Introduction

The purpose of this document is to evaluate potential security weaknesses and implement security controls within the mobile app's infrastructure.

Scope of Work

This exercise focuses on 1 user story within the Zentra Bank mobile app:

1. As a user, I want to **receive push notifications** so that I stay informed about important updates in my account

Out of Scope

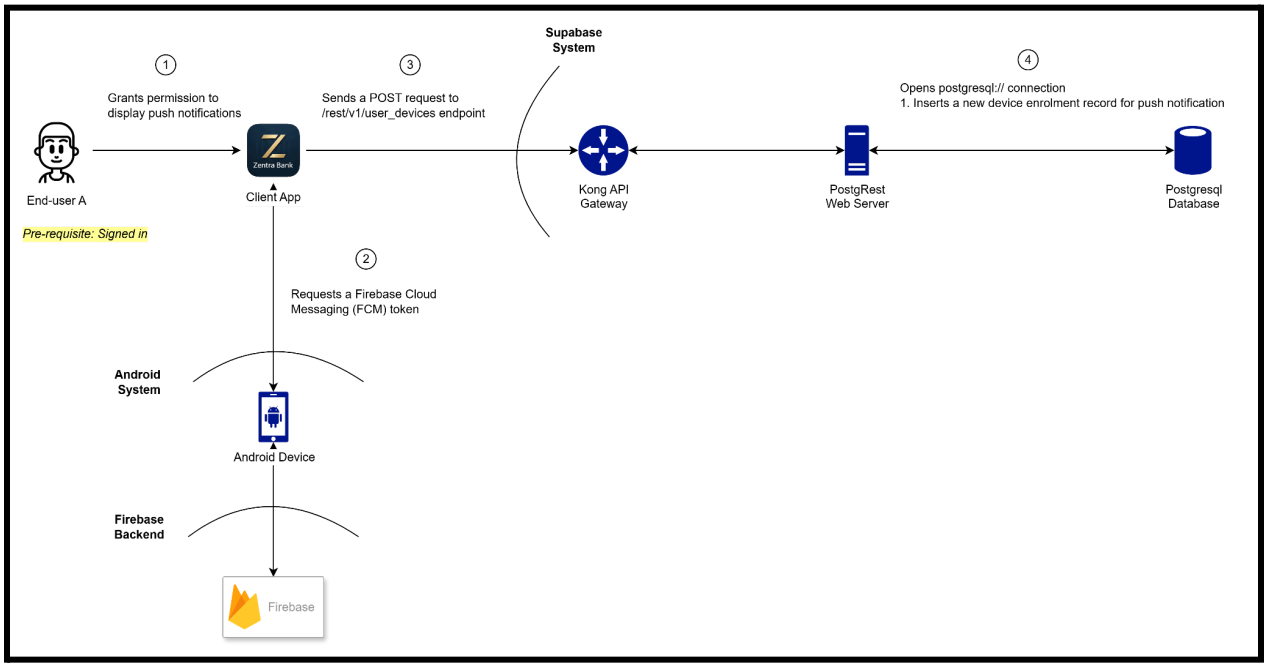
This exercise strictly focuses on the mobile application and its functionalities. Any components outside the mobile app, which includes but not limited to REST APIs, server infrastructure, remote database, and external services, are not within the scope.

User story 6

The data flow for this user story consists of 2 key stages: 1) End-user A enables push notification and waits to receive one. 2) End-user B transfers money to End-user A, which triggers that notification. The detailed steps are as follows:

"As a user, I want to receive push notifications so that I stay informed about important updates in my account"

<u>1) End-user A enables push notification and waits to receive one.</u>

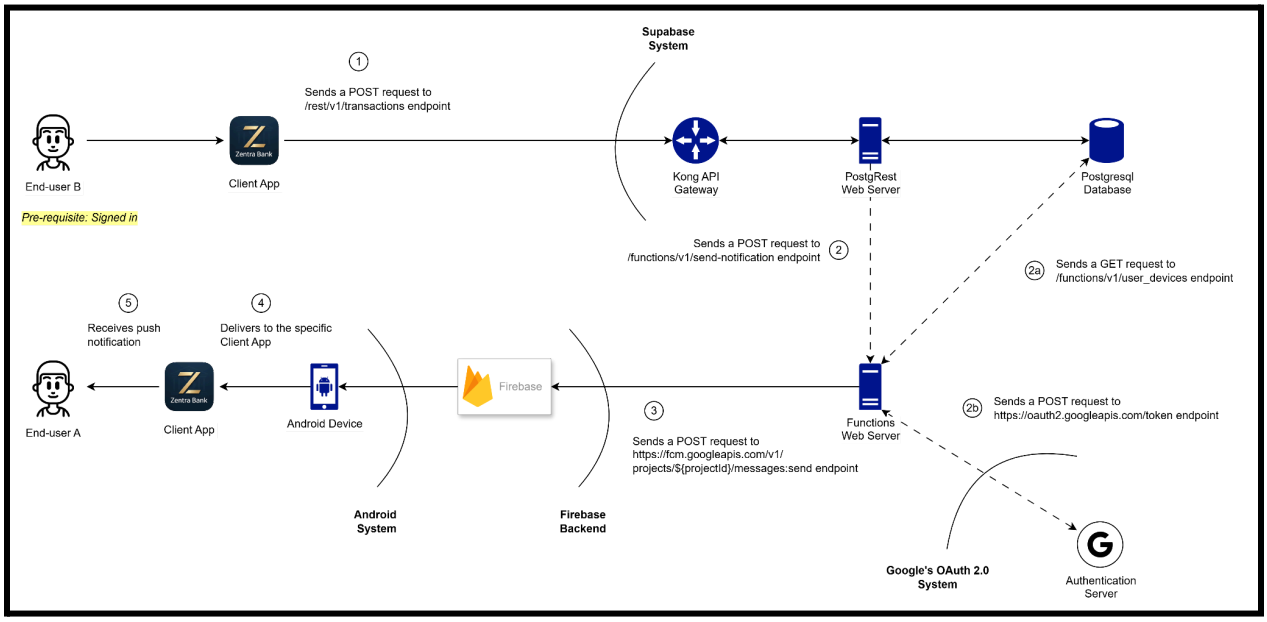


1. The End-user A grants the Client App (CA) permission to display push notifications
2. The Client App (CA) requests a Firebase Cloud Messaging (FCM) registration token via the Firebase SDK, which internally forwards the request to Google Play Services that communicates with the FCM backend
    a. A FCM registration token is a unique string that identifies each client app instance
3. The Client App (CA) receives the FCM registration token and makes an insert request via REST API to the Web server

```
REQUEST:
https://vomohccmapvpspaupbah.supabase.co/rest/v1/user_devices?on_conflict=user_id%2Cplatform
{"body":
"{\"user_id\":\"a9b643b3-9644-43acc-ae74-cc8a14b821bc\",\"account_id\":\"6e7084a4-da6c-48eb-bd16-6f
9f50de1c5e\",\"fcm_token\":\"f6OonhSMR4-T12_7BD_EwM:APA91bHQ3p7_lY07KQD7WcDxxxdJsJPHoxoNBNHzunt5e9F
zOzxadYj2bYCxqjFjjpYkhLXfptcLkbYxSpzDvdv_S25s_5epISM4h3t2n-B18Nq2egtGP8g\",\"platform\":\"android\"
,\"is_active\":true,\"updated_at\":\"2025-09-17T01:34:51.239Z\"}", "headers": {"map": {"apikey":
"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpc3MiOiJzdXBhYmFzZSIsInJlZiI6InZvbW9oY2NtYXB2cHNwYXVwYmFoI
iwicm9sZSI6ImFub24iLCJpYXQiOjE3NDg0ODA2MjQsImV4cCI6MjA2NDA1NjYyNH0.kAtm3_e8PUjQGsacDTt7nzrBmQ49H-wU
4J4_biPMSZQ", "authorization": "Bearer
eyJhbGciOiJIUzI1NiIsImtpZCI6Ik4vampuNklBTGRLeHZpRmUiLCJ0eXAiOiJKV1QifQ.eyJpc3MiOiJodHRwczovL3ZvbW9o
Y2NtYXB2cHNwYXVwYmFoLnN1cGFiYXNlLmNvL2F1dGgvdjEiLCJzdWIiOiJhOWI2NDNiMy05NjQ0LTQzYWMtYWU3NC1jYzhhMTR
iODIxYmMiLCJhdWQiOiJhdXRoZW50aWNhdGVkIiwiZXhwIjoxNzU4MDc2NDkwLCJpYXQiOjE3NTgwNzI4OTAsImVtYWlsIjoidG
VzdEB0ZXN0LmNvbSIsInBob25lIjoiIiwiYXBwX21ldGFkYXRhIjp7InByb3ZpZGVyIjoiZW1haWwiLCJwcm92aWRlcnMiOlsiZ
W1haWwiXX0sInVzZXJfbWV0YWRhdGEiOnsiZW1haWwiOiJ0ZXN0QHRlc3QuY29tIiwiZW1haWxfdmVyaWZpZWQiOnRydWUsInBo
b25lX3Zlcmlmaw VkIjpmYWxzZSwic3ViIjoiYTliNjQzYjMtOTY0NC00M2FjLWFlNzQtY2M4YTE0YjgyMWJjIn0sInJvbGUiOiJ
hdXRoZW50aWNhdGVkIiwiYWFsIjoiYWFsMSIsImFtciI6W3sibWV0aG9kIjoicGFzc3dvcmQiLCJ0aW1lc3RhbXAiOjE3NTgwNz
I4OTB9XSwic2Vzc2lvbl9pZCI6ImM3MDRiMDk4LTFhM2UtNDMxMi1iMjc1LTYwY2VlN2QzOTBlZSIsImlzX2Fub255bW91cyI6Z
mFsc2V9.XijKeWPbjwJlooCZEb0U7VQxBX3P4yvHAcAhS1ZBvGc", "content-profile": "public", "content-type":
"application/json", "prefer": "resolution=merge-duplicates", "x-client-info":
"supabase-js-react-native/2.49.5-next.1"}}, "method": "POST", "signal": undefined}
```


4. The Web Server (WS) makes an insert query to **`user_devices`** table in the Database (DB) to store the FCM registration token

2) End-user B transfers money to End-user A, which triggers that notification.



1. The End-user B inputs End-user A account number (see User Story 4)

a. To complete the transfer, the Client App (CA) sends a series of requests via REST API to the Web server, specifically inserting a new transaction record to the **`transactions`** table in the Database (DB) via Postgres connection


```
REQUEST:
https://vomohccmapvpspaupbah.supabase.co/rest/v1/transactions?columns=%22account_id%22%2C%22booking
_datetime%22%2C%22amount%22%2C%22currrency%22%2C%22credit_debit%22%2C%22description%22%2C%22transac
tion_type%22%2C%22status%22 {"body":
"[{\"account_id\":\"6e7084a4-da6c-48eb-bd16-6f9f50de1c5e\",\"booking_datetime\":\"2025-09-17T01:40:
17.448Z\",\"amount\":50,\"currency\":\"SGD\",\"credit_debit\":\"Debit\",\"description\":\"Transfer
to 00153940154\",\"transaction_type\":\"Online
Transfer\",\"status\":\"Booked\"},{\"account_id\":\"7769d110-7ee6-4c32-a5ef-a06f59ed6fce\",\"bookin
g_datetime\":\"2025-09-17T01:40:17.448Z\",\"amount\":50,\"currency\":\"SGD\",\"credit_debit\":\"Cre
dit\",\"description\":\"Transfer from 00162624385\",\"transaction_type\":\"Online
Transfer\",\"status\":\"Booked\"}]", "headers": {"map": {"apikey":
"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpc3MiOiJzdXBhYmFzZSIsInJlZiI6InZvbW9oY2NtYXB2cHNwYXVwYmFoI
iwicm9sZSI6ImFub24iLCJpYXQiOjE3NDg0ODA2MjQsImV4cCI6MjA2NDA1NjYyNH0.kAtm3_e8PUjQGsacDTt7nzrBmQ49H-wU
4J4_biPMSZQ", "authorization": "Bearer
eyJhbGciOiJIUzI1NiIsImtpZCI6Ik4vampuNklBTGRLeHZpRmUiLCJ0eXAiOiJKV1QifQ.eyJpc3MiOiJodHRwczovL3ZvbW9o
Y2NtYXB2cHNwYXVwYmFoLnN1cGFiYXNlLmNvL2F1dGgvdjEiLCJzdWIiOiJhOWI2NDNiMy05NjQ0LTQzYWMtYWU3NC1jYzhhMTR
iODIxYmMiLCJhdWQiOiJhdXRoZW50aWNhdGVkIiwiZXhwIjoxNzU4MDc2NDkwLCJpYXQiOjE3NTgwNzI4OTAsImVtYWlsIjoidG
VzdEB0ZXN0LmNvbSIsInBob25lIjoiIiwiYXBwX21ldGFkYXRhIjp7InByb3ZpZGVyIjoiZW1haWwiLCJwcm92aWRlcnMiOlsiZ
W1haWwiXX0sInVzZXJfbWV0YWRhdGEiOnsiZW1haWwiOiJ0ZXN0QHRlc3QuY29tIiwiZW1haWxfdmVyaWZpZWQiOnRydWUsInBo
b25lX3ZlcmlmaWVkIjpmYWxzZSwic3ViIjoiYTliNjQzYjMtOTY0NC00M2FjLWFlNzQtY2M4YTE0YjgyMWJjIn0sInJvbGUiOiJ
hdXRoZW50aWNhdGVkIiwiYWFsIjoiYWFsMSIsImFtciI6W3sibWV0aG9kIjoicGFzc3dvcmQiLCJ0aW1lc3RhbXAiOjE3NTgwNz
I4OTB9XSwic2Vzc2lvbl9pZCI6ImM3MDRiMDk4LTFhM2UtNDMxMi1iMjc1LTYwY2VlN2QzOTBlZSIsImlzX2Fub255bW91cyI6Z
mFsc2V9.XijKeWPbjwJlooCZEb0U7VQxBX3P4yvHAcAhS1ZBvGc", "content-profile": "public", "content-type":
"application/json", "prefer": "", "x-client-info": "supabase-js-react-native/2.49.5-next.1"}},
"method": "POST", "signal": undefined}
```


2. Upon the new insert, the PostgRest Web Server (WS) will send a request to inform the new transaction record via REST API to Functions Web Server (WS) - a serverless function

   a. The serverless function first retrieves the FCM registration token via REST API to the Web server, and accesses the **`user_devices`** table in the Database (DB) via Postgres connection.

   b. The serverless function then requests for an access token via REST API to Google's OAuth 2.0 for authorisation

```
Request to serverless function:
{
  "event_message": "Request =  Request {\n  bodyUsed: false,\n  headers: Headers {\n    accept: \"*/*\",\n
  \"accept-encoding\": \"gzip\",\n    authorization: \"Bearer
  eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpc3MiOiJzdXBhYmFzZSIsInJlZiI6InZvbW9oY2NtYXB2cHNwYXVwYmFoIiwicm9sZSI6InNlcn
  ZpY2Vfcm9sZSIsImlhdCI6MTc0ODQ4MDYyNCwiZXhwIjoyMDY0MDU2NjI0fQ.WcUibQGOXF8h9jFXdeoU9ZnmMReDOKptVQq99WWrUSs\",\n
  baggage: \"sb-request-id=01995554-67d8-76f3-9918-df0e612cd367\",\n    \"cdn-loop\": \"cloudflare; loops=1;
  subreqs=1\",\n    \"cf-connecting-ip\": \"52.77.5.167\",\n    \"cf-ew-via\": \"15\",\n    \"cf-ray\":
  \"9804e51302366590-SIN\",\n    \"cf-visitor\": '{\"scheme\":\"https\"}',\n    \"cf-worker\": \"supabase.co\",\n
  \"content-length\": \"310\",\n    \"content-type\": \"application/json\",\n    host:
  \"edge-runtime.supabase.com\",\n    \"sb-request-id\": \"01995554-67d8-76f3-9918-df0e612cd367\",\n
  \"user-agent\": \"pg_net/0.14.0\",\n    \"x-amzn-trace-id\": \"Root=1-68ca1183-2f4ac7056ae6244e24a56a08\",\n
  \"x-forwarded-for\": \"52.77.5.167,52.77.5.167, 13.248.127.236\",\n    \"x-forwarded-port\": \"443\",\n
  \"x-forwarded-proto\": \"https\"\n  },\n  method: \"POST\",\n  redirect: \"follow\",\n  url:
  \"http://vomohccmapvpspaupbah.supabase.co/send-notification\"\n}\n",
  "id": "d0466fc2-f86b-4910-bcb8-86c3b7fa9f48",
  "metadata": [
    {
      "boot_time": null,
      "cpu_time_used": null,
      "deployment_id": "vomohccmapvpspaupbah_4c70ad3d-1983-4328-a631-a7f3c4f9c04f_5",
      "event_type": "Log",
      "execution_id": "27850e28-78e2-4656-ac14-5f2da482b011",
      "function_id": "4c70ad3d-1983-4328-a631-a7f3c4f9c04f",
      "level": "info",
      "memory_used": [],
      "project_ref": "vomohccmapvpspaupbah",
      "reason": null,
      "region": "ap-southeast-1",
      "served_by": "supabase-edge-runtime-1.69.4 (compatible with Deno v2.1.4)",
      "timestamp": "2025-09-17T01:40:19.182Z",
      "version": "5"
    }
  ],
  "timestamp": 1758073219182000
}
```


3. The serverless function sends a request consisting the FCM registration token via REST API to FCM backend with the access token to deliver the push notification to the Google Play services of End-user A device

4. The Google Play services of End-user A device then delivers that request to the specific Client App (CA)

5. The Client App of End-user A's device receives the push notification

Reference: https://firebase.google.com/docs/cloud-messaging/fcm-architecture

Methodology

The methodology for this exercise follows a structured approach:

1. Create data flow diagrams to map how data moves within the mobile app
2. Use security testing tools and threat modeling to identify vulnerabilities
    a. Mobile Application Security Framework (MobSF)

3. Evaluate the impact of the identified vulnerability, how they could affect the app
4. Research remediation measures and show their effectiveness

Findings Overview

US06-MOBSF01. My app uses firebase cloud messaging for push notification. Attackers can potentially use exfiltrated data collected by firebase analytics for surveillance, profiling, or targeted attacks..... 17

US06-MOBSF01-C01. Remove threat by ensuring the app excludes unnecessary or unused libraries like trackers..... 18

Template

Legend
- White background cells: For interns to fill
- Blue / Orange / Green background cells: For Supervising officer to fill

Issue: App can be installed on a vulnerable unpatched Android version
Issue ID: US01-MOBSF01. My app allows installation by devices running on Android 7.0 and above. Users on older versions without security updates are exposed to known, unpatched vulnerabilities, which attackers can exploit. **Cause**: My app allows installation by devices running on Android 7.0 and above. **Effect**: Users on older versions without security updates are exposed to known, unpatched vulnerabilities, which attackers can exploit.
Demonstration: - Show what this issue means to the App. - Explain with evidence on what this issue means to the App?
Control ID: US01-MOBSF01-C01. Remove threat by ensuring the app installs only on devices running Android versions with active security support. **Control**: Remove threat by ensuring the app installs only on devices running Android versions with active security support.
Implemented at: Client App / Authentication Server / Database **Demonstration**: - Show how the control mitigates the threat. - Explain with evidence on how you implement the control?
Status: tried (implemented) / tested (tested for effectiveness)
Another implementation

Implemented at:
Client App / Authentication Server / Database

Demonstration:
- Show how the control mitigates the threat.
- Explain with evidence on how you implement the control?

Status:
tried (implemented) / tested (tested for effectiveness)

Control ID:
US01-MOBSF01-C02. Reduce threat by ensuring the app detects if it's running on Android versions without active security support.

Control:
Reduce threat by ensuring the app detects if it's running on Android versions without active security support.

Implemented at:
Client App / Authentication Server / Database

Demonstration:
- Show how the control mitigates the threat.
- Explain with evidence on how you implement the control?

Status:
tried (implemented) / tested (tested for effectiveness)

New risks that arise as a result of implementing user stories

Issue: Broadcast Receiver (io.invertase.firebase.messaging.ReactNativeFirebaseMessagingReceiver) is Protected by a permission, but the protection level of the permission should be checked.
Demonstration: - Show what this issue means to the App. - Explain with evidence on what this issue means to the App? This is the specific line that was flagged out by MobSF: `com.google.android.c2dm.permission.SEND [android:exported=true]` Note: The permission was added because we incorporated Firebase SDK. This is a similar issue that has already been resolved in previous findings. Controls include using an appropriate protection level, validating the identity of the caller against a trusted whitelist and setting the android:exported attribute to "false" for components not intended to be accessible by other apps. More information A Broadcast Receiver is found to be shared with other apps on the device therefore leaving it accessible to any other application on the device. It is protected by a permission which is not defined in the analysed application. As a result, the protection level of the permission should be checked where it is defined. If it is set to normal or dangerous, a malicious application can request and obtain the permission and interact with the component. If it is set to signature, only applications signed with the same certificate can obtain the permission. https://github.com/invertase/react-native-firebase/blob/main/packages/messaging/android/src/main/java/io/invertase/firebase/messaging/ReactNativeFirebaseMessagingReceiver.java

Issue: Broadcast Receiver (com.google.firebase.iid.FirebaseInstanceIdReceiver) is Protected by a permission, but the protection level of the permission should be checked.

Demonstration:
- Show what this issue means to the App.
- Explain with evidence on what this issue means to the App?

This is the specific line that was flagged out by MobSF:
`com.google.android.c2dm.permission.SEND [android:exported=true]`

Note: The permission was added because we incorporated Firebase SDK.

This is a similar issue that has already been resolved in previous findings. Controls include using an appropriate protection level, validating the identity of the caller against a trusted whitelist and setting the android:exported attribute to "false" for components not intended to be accessible by other apps.

More information
A Broadcast Receiver is found to be shared with other apps on the device therefore leaving it accessible to any other application on the device. It is protected by a permission which is not defined in the analysed application. As a result, the protection level of the permission should be checked where it is defined. If it is set to normal or dangerous, a malicious application can request and obtain the permission and interact with the component. If it is set to signature, only applications signed with the same certificate can obtain the permission.

https://firebase.google.com/docs/reference/android/com/google/firebase/iid/FirebaseInstanceIdReceiver#FirebaseInstanceIdReceiver()

Issue: Service (com.google.android.gms.auth.api.signin.RevocationBoundService) is Protected by a permission, but the protection level of the permission should be checked.

Demonstration:
- Show what this issue means to the App.
- Explain with evidence on what this issue means to the App?

This is the specific line that was flagged out by MobSF:
`com.google. android.gms.auth.api.signin.permission.REVOCATION_NOTIFICATION [android:exported=true]`

Note: The permission was added because we incorporated Firebase SDK.

This is a similar issue that has already been resolved in previous findings. Controls include using an appropriate protection level, validating the identity of the caller against a trusted whitelist and setting the android:exported attribute to "false" for components not intended to be accessible by other apps.

More information
A Service is found to be shared with other apps on the device therefore leaving it accessible to any other application on the device. It is protected by a permission which is not defined in the analysed application. As a result, the protection level of the permission should be checked where it is defined. If it is set to normal or dangerous, a malicious application can request and obtain the permission and interact with the component. If it is set to signature, only applications signed with the same certificate can obtain the permission.

https://developers.google.com/android/reference/kotlin/com/google/android/gms/auth/api/signin/package-summary

https://stackoverflow.com/questions/14548927/android-broadcast-receiver-vs-service

Issue: Files may contain hardcoded sensitive information like usernames, passwords, keys etc. (False positive)

Demonstration:
- Show what this issue means to the App.
- Explain with evidence on what this issue means to the App?

These are the specific lines that was flagged out by MobSF:
io/invertase/firebase/common/TaskExecutorService.java, line(s) 15,16
https://github.com/invertase/react-native-firebase/blob/main/packages/app/android/src/main/java/io/invertase/firebase/common/TaskExecutorService.java

```
31  public class TaskExecutorService {
32    private static final String MAXIMUM_POOL_SIZE_KEY = "android_task_executor_maximum_pool_size";
33    private static final String KEEP_ALIVE_SECONDS_KEY = "android_task_executor_keep_alive_seconds";
34
```

```
42      ReactNativeFirebaseJSON json = ReactNativeFirebaseJSON.getSharedInstance();
43      this.maximumPoolSize = json.getIntValue(MAXIMUM_POOL_SIZE_KEY, 1);
44      this.keepAliveSeconds = json.getIntValue(KEEP_ALIVE_SECONDS_KEY, 3);
```

io/invertase/firebase/messaging/ReactNativeFirebaseMessagingHeadlessService.java, line(s) 15,13
https://github.com/invertase/react-native-firebase/blob/main/packages/messaging/android/src/main/java/io/invertase/firebase/messaging/ReactNativeFirebaseMessagingHeadlessService.java

```
11  public class ReactNativeFirebaseMessagingHeadlessService extends HeadlessJsTaskService {
12    private static final long TIMEOUT_DEFAULT = 60000;
13    private static final String TIMEOUT_JSON_KEY = "messaging_android_headless_task_timeout";
14    private static final String TASK_KEY = "ReactNativeFirebaseMessagingHeadlessTask";
```

```
22      return new HeadlessJsTaskConfig(
23          TASK_KEY,
24          ReactNativeFirebaseMessagingSerializer.remoteMessageToWritableMap(remoteMessage),
25          ReactNativeFirebaseJSON.getSharedInstance().getLongValue(TIMEOUT_JSON_KEY, TIMEOUT_DEFAULT),
26          // Prevents race condition where the user opens the app at the same time as a notification
```

io/invertase/firebase/messaging/ReactNativeFirebaseMessagingSerializer.java, line(s) 21
https://github.com/invertase/react-native-firebase/blob/main/packages/messaging/android/src/main/java/io/invertase/firebase/messaging/ReactNativeFirebaseMessagingSerializer.java

```java
public class ReactNativeFirebaseMessagingSerializer {
  private static final String KEY_TOKEN = "token";
  private static final String KEY_COLLAPSE_KEY = "collapseKey";
  private static final String KEY_DATA = "data";
  private static final String KEY_FROM = "from";
  private static final String KEY_MESSAGE_ID = "messageId";
  private static final String KEY_MESSAGE_TYPE = "messageType";
  private static final String KEY_SENT_TIME = "sentTime";
  private static final String KEY_ERROR = "error";
  private static final String KEY_TO = "to";
  private static final String KEY_TTL = "ttl";
  private static final String KEY_PRIORITY = "priority";
  private static final String KEY_ORIGINAL_PRIORITY = "originalPriority";
```

These are not an issue to the App, because none of the flagged variables actually store real keys. These matches occurred because the scanning uses a regex that detects the pattern “KEY” without considering whether the variable truly contains a secret or sensitive value. (See below screenshot).

RULE

Expand rule

```yaml
rules:
  - id: hardcoded_api_key
    patterns:
      - pattern-not: $X = "";
      - pattern-not: $M($X, "", ...);
      - pattern-either:
          - pattern: |
              $X = "...";
          - pattern: |
              $M($X, "...", ...);
      - metavariable-regex:
          metavariable: $X
```

TEST CODE

```java
private static final String MAXIMUM_POOL_SIZE_KEY =
"android_task_executor_maximum_pool_size";
```

OUTPUT

L1: A hardcoded Key is identified.

All tests pass

Run

Issue: App can read/write to External Storage. Any app can read data written to External Storage. (False positive)

Demonstration:
- Show what this issue means to the App.
- Explain with evidence on what this issue means to the App?

This is the specific file that was flagged out by MobSF:
io/invertase/firebase/utils/ReactNativeFirebaseUtilsModule.java, line(s) 114,123,124, 125
https://github.com/invertase/react-native-firebase/blob/main/packages/app/android/src/reactnative/java/io/invertase/firebase/utils/ReactNativeFirebaseUtilsModule.java

```
151         constants.put(KEY_CACHE_DIRECTORY, context.getCacheDir().getAbsolutePath());
152
153         File externalDirectory = context.getExternalFilesDir(null);
154         if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.KITKAT) {
155           if (externalDirectory != null) {
```

```
170       constants.put(
171           KEY_PICS_DIRECTORY,
172           Environment.getExternalStoragePublicDirectory(Environment.DIRECTORY_PICTURES)
173               .getAbsolutePath());
174
175       constants.put(
176           KEY_MOVIES_DIRECTORY,
177           Environment.getExternalStoragePublicDirectory(Environment.DIRECTORY_MOVIES)
178               .getAbsolutePath());
179
180       File externalStorageDirectory = Environment.getExternalStorageDirectory();
181       if (externalStorageDirectory != null) {
182         constants.put(KEY_EXT_STORAGE_DIRECTORY, externalStorageDirectory.getAbsolutePath());
183       }
```

```
118       ref: https://github.com/MobSF/owasp-mstg/blob/master/Document/0x05h-Testing-Platform-Interaction
119     - id: android_read_write_external
120       message: >-
121         App can read/write to External Storage. Any App can read data written to
122         External Storage.
123       type: RegexOr
124       pattern:
125         - \.getExternalStorage
126         - \.getExternalFilesDir\(
127       severity: warning
```

This is not an issue to the App, because the flagged functions only returns a map of useful paths including external file directories, it does not read/write to them. It also does not mean the app has read/write access to them.

More information
.getExternalFilesDir()
https://developer.android.com/training/data-storage/app-specific
An external storage directory that is private to your application. Files stored here are accessible only by your app and are not typically visible to the user as media. These files will be automatically deleted when the application is uninstalled. Importantly, no special permissions are required to read or write to this directory, and other apps cannot access its contents directly.

.getExternalStoragePublicDirectory()

A shared public directory like Pictures where users typically place and manage their own files. Unlike app-specific directories, these locations are accessible by other apps and the user. Because of this, your app must request runtime permissions like `READ_EXTERNAL_STORAGE` or `WRITE_EXTERNAL_STORAGE` to interact with these folders, especially on Android versions before 10. On Android 10 and above, access is further restricted by scoped storage, requiring the use of system APIs like MediaStore or the Storage Access Framework for file operations.

.getExternalStorageDirectory()
The root of the primary shared or external storage volume. Traditionally, this was an SD card but nowadays it's often the main shared storage partition on the device, such as `/storage/emulated/0`. This directory can be mounted to a computer for file transfers. However, access to this root directory is heavily restricted on newer Android versions due to scoped storage enforcement. Direct reading or writing here is generally discouraged and requires special permissions and legacy flags.

Issue: Application contains Privacy Trackers.

Issue ID:
US06-MOBSF01. My app uses firebase cloud messaging for push notification. Attackers can potentially use exfiltrated data collected by firebase analytics for surveillance, profiling, or targeted attacks.

Cause:
My app uses firebase cloud messaging for push notification.

Effect:
Attackers can potentially use exfiltrated data collected by firebase analytics for surveillance, profiling, or targeted attacks.

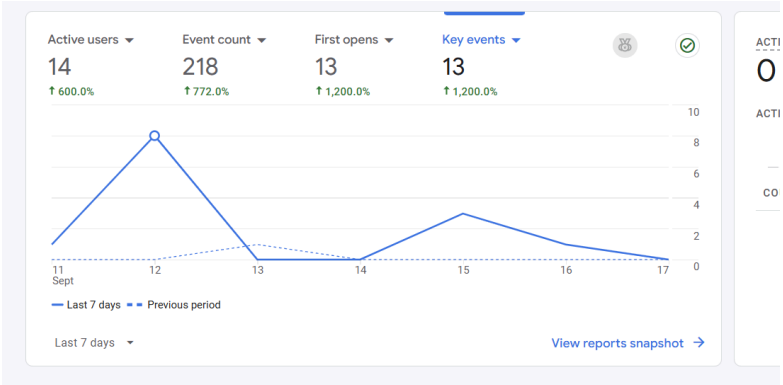
Demonstration:
- Show what this issue means to the App.
- Explain with evidence on what this issue means to the App?

MobSF flagged the use of 1 privacy tracker: Google Firebase Analytics
This is an issue to the App, because data collection is happening without explicit developer consent. I am going to show that using firebase cloud messaging allows data collection on firebase analytics.

The APK uses React Native firebase cloud messaging. When installing, the firebase analytics npm package is downloaded too.

```
PS D:\Documents\android-mobile-banking-app> npm ls
mobile-banking-app@1.0.0 D:\Documents\android-mobile-banking-app
├── @babel/core@7.27.1
├── @react-native-async-storage/async-storage@2.1.2
├── @react-native-firebase/analytics@23.1.2
├── @react-native-firebase/app@23.1.2
├── @react-native-firebase/messaging@23.1.2
├── @shopify/react-native-skia@2.0.0-next.4
├── @supabase/supabase-js@2.49.5-next.1
├── @types/react@19.0.14
├── babel-plugin-dotenv@0.1.1
├── cross-fetch@4.1.0
├── expo-constants@17.1.6
├── expo-dev-client@5.2.1
├── expo-router@5.0.7
├── expo-status-bar@2.2.3
├── expo@53.0.9
├── react-native-dotenv@3.4.11
├── react-native-qrcode-svg@6.3.15
├── react-native-reanimated@3.17.5
├── react-native-safe-area-context@5.4.0
├── react-native-screens@4.10.0
├── react-native-svg@15.11.2
├── react-native-url-polyfill@2.0.0
├── react-native-vision-camera@4.7.2
├── react-native@0.79.2
├── react@19.0.0
└── typescript@5.8.3
```

The firebase analytics shows the data collection on their console. Notice the active users “14”, showing data collection from the app.



More information
Tracker name: Google Firebase Analytics
https://reports.exodus-privacy.eu.org/en/trackers/49/
https://support.google.com/firebase/answer/6318039?hl=en&ref_topic=6386699&sjid=10849990332744961234-NC

A tracker is a piece of software whose task is to gather information on the person using the application, on how they use it, or on the smartphone being used. A tracker is usually distributed by companies as a SDK (Software Development Kit), a sort of ready-made toolkit, aiming to make it easier for application developers. Known usages: crash reports, analytics, profiling, identification, ads, location
https://reports.exodus-privacy.eu.org/en/info/trackers/

Control ID:
US06-MOBSF01-C01. Remove threat by ensuring the app excludes unnecessary or unused libraries like trackers.

Control:
Remove threat by ensuring the app excludes unnecessary or unused libraries like trackers.

Implemented at:
Client App

Demonstration:
- Show how the control mitigates the threat.
- Explain with evidence on how you implement the control?

I am going to show that Firebase Analytics data collection can be removed without affecting other Firebase cloud messaging features.

1. I will uninstall the firebase analytics independently using npm commands
2. I expect that the tracker is removed from the apk
3. I expect that firebase analytics console will no longer show data

---

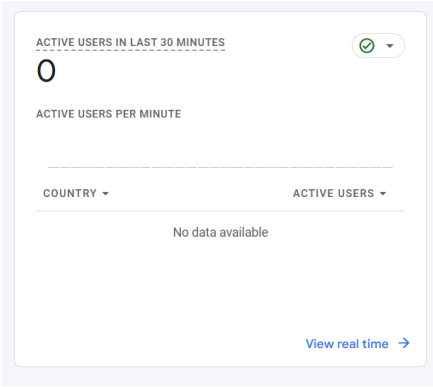
Step 1: Uninstall the npm package from the node module

```
PS D:\Documents\android-mobile-banking-app> npm uninstall @react-native-firebase/analytics

removed 2 packages, and audited 839 packages in 3s

73 packages are looking for funding
  run `npm fund` for details

2 low severity vulnerabilities

To address all issues, run:
  npm audit fix

Run `npm audit` for details.
```

Step 2: The tracker is now removed from the apk

```
PS D:\Documents\android-mobile-banking-app> npm ls
mobile-banking-app@1.0.0 D:\Documents\android-mobile-banking-app
├── @babel/core@7.27.1
├── @react-native-async-storage/async-storage@2.1.2
├── @react-native-firebase/app@23.1.2
├── @react-native-firebase/messaging@23.1.2
├── @shopify/react-native-skia@2.0.0-next.4
├── @supabase/supabase-js@2.49.5-next.1
├── @types/react@19.0.14
├── babel-plugin-dotenv@0.1.1
├── cross-fetch@4.1.0
├── expo-constants@17.1.6
├── expo-dev-client@5.2.1
├── expo-router@5.0.7
├── expo-status-bar@2.2.3
├── expo@53.0.9
├── react-native-dotenv@3.4.11
├── react-native-qrcode-svg@6.3.15
├── react-native-reanimated@3.17.5
├── react-native-safe-area-context@5.4.0
├── react-native-screens@4.10.0
├── react-native-svg@15.11.2
├── react-native-url-polyfill@2.0.0
├── react-native-vision-camera@4.7.2
├── react-native@0.79.2
├── react@19.0.0
└── typescript@5.8.3
```

Step 3: Observe that the console shows the active users fall from “14” to “0”. Showing that the console is no longer collecting information from the app.



References

https://tuta.com/blog/app-tracking

Status:
tested (tested for effectiveness)

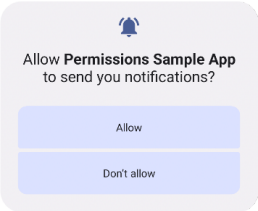
Issue: android.permission.POST_NOTIFICATIONS (dangerous): allows an app to post notifications. - Allows an app to post notifications (Critical permissions)

Demonstration:
- Show what this issue means to the App.
- Explain with evidence on what this issue means to the App?

This is the specific permission that was flagged out by MobSF:
`android.permission.POST_NOTIFICATIONS`

This is not an issue to the App as long as it is used responsibly. Problems only arise if the app misuses the permission to spam users with excessive notifications or tries to trick users into clicking phishing links, which is typically seen in malicious apps. Since the app requires this permission to send push notifications, its presence is justified.

More information
This permission only exists on Android 13. When the app is running on devices with Android 12 and lower, POST_NOTIFICATIONS permission is not needed. On Android 13, some notifications can still be displayed without this permission, such as notifications for foreground services or media sessions.



Common misuse:
Aggressively pushing advertising onto users with push notification, regardless of the fact that users will not be persuaded by the ads since they are so annoying. The purpose is not for you to be convinced by the ads, but for the hackers to exploit a pay-per-view advertising program.
https://heimdalsecurity.com/blog/push-notifications-security-risks-how-to-disable/

Directing users towards scam websites with push notification. This way, some users even fell for scams and entered sensitive info on phishing forms which were mimicking legit email and banking service sites.
https://www.androidpolice.com/android-notifications-potential-open-link-exploit/